Acceptance Testing 2.1: Number Guessing Game

To do: Make a submission **To do:** Receive a grade

Opened: Tuesday, 1 October 2024, 10:00 AM
Due: Tuesday, 8 October 2024, 10:00 AM

Overview

In this assignment, you will write acceptance tests for the number guessing game application using Robot Framework. The game has both a web and CLI version, but you will focus on the web version. You are already familiar with the game, which involves choosing a difficulty level, placing bets, and guessing numbers. The top ten high scores are saved to a scoreboard.

This time, you'll focus on writing tests based on user stories to validate the functionality of the game, such as tracking high scores on the scoreboard and ensuring users can play the game as expected.

Start by downloading the game source code, which is attached to this assignment (see below).

Game Rules

- You start with 100 bet points.
- Choose a difficulty level:
  - Easy: 10 guesses
  - Hard: 5 guesses
- Place a bet with your bet points for each round.
- Guess the number between 1 and 100:
  - If you guess correctly, you can choose to double your prize or take your winnings.
  - If you choose to double, you have a 50% chance to either double your prize or lose it.
- If you run out of guesses, you can try to save your bet:
  - You have a 50% chance to keep your bet points or lose the bet points you placed for that round.
- After each round, you can decide to continue playing or finish the game and save your score to the scoreboard.
- The game ends when you choose to finish or you have no bet points left.
- The high scores are saved and displayed on the scoreboard. Try to get your name in the top 10!

How to Play the Game

Web Version:

- Run the Web Application:
  - Open a terminal and navigate to the project directory.
  - Run the following command: `python app.py`
  - Open your web browser and go to `http://127.0.0.1:5000/`.

CLI Version:

- Run the CLI Application:
  - Open a terminal and navigate to the project directory.
  - Run the following command: `python game.py`
  - Follow the prompts:
    - Enter your name.
    - Choose a difficulty level.
    - Place bets and guess numbers as prompted in the terminal.

How to Install Dependencies

- Ensure you have Python installed on your system.
- Open a terminal and navigate to the project directory.
- Run the following command to install the required dependencies:

```
pip install -r requirements.txt
```

Part 1: Full Game Playthrough with Valid Inputs

In this part, you will write a detailed acceptance test case that simulates a full game playthrough from start to finish using valid inputs. The test will cover starting the game, entering a name, selecting a difficulty level, placing bets, making guesses, and ensuring the game handles correct and incorrect guesses appropriately.

Scenarios to Implement

- **Normal Game Playthrough**
  - Stimulus: The user opens the game in a browser.
  - Response: The application displays the main game page.
  - Stimulus: The user enters a valid player name and starts the game.
  - Response: The game proceeds to the difficulty selection page.
  - Stimulus: The user selects a valid difficulty level.
  - Response: The game proceeds to the betting page.
  - Stimulus: The user places a valid bet.
  - Response: The game proceeds to the guessing page.
  - Stimulus: The user makes valid guesses to find the correct number efficiently.
  - Response: The game provides feedback after each guess and accepts the correct guess.
  - Stimulus: The user chooses to take the award without doubling.
  - Response: The game updates the bet points accordingly.
  - Stimulus: The user decides to finish the game.
  - Response: The game shows a "Congratulations" message.

Note: In the Easy difficulty mode, you have enough guesses to find the correct number using binary search. To make efficient guesses, you can use our provided implementation of binary search. Check the hints section for more information.

- **Repetitive Incorrect Guesses Leading to Loss**
  - Stimulus: The user opens the game in a browser.
  - Response: The application displays the main game page.
  - Stimulus: The user enters a valid player name and starts the game.
  - Response: The game proceeds to the difficulty selection page.
  - Stimulus: The user selects a valid difficulty level.
  - Response: The game proceeds to the betting page.
  - Stimulus: The user places all of their bets.
  - Response: The game proceeds to the guessing page.
  - Stimulus: The user repeatedly enters the same incorrect number until guesses run out.
  - Response: The game decreases the remaining guesses and eventually displays an "Out of Guesses" message.
  - Stimulus: The user doesn't attempt to save the bet.
  - Response: The game updates the bet points accordingly, and shows a "Game Over" message.

Part 2: Invalid Input Testing

In this part, you will write detailed acceptance test cases to ensure the number guessing game handles invalid inputs appropriately. You can utilize the Set Session Parameters helper keyword defined in `helpers.robot` to jump to specific parts of the game for efficient and targeted testing.

Scenarios to Implement

- **Invalid Name Input**
  - Stimulus: The user opens the game in a browser.
  - Response: The application displays the main game page.
  - Stimulus: The user enters an invalid player name (e.g., empty string, special characters).
  - Response: The game displays an error message and does not proceed.
- **Invalid Difficulty Selection**
  - Stimulus: The user starts the game with a valid player name.
  - Response: The game proceeds to the difficulty selection page.
  - Stimulus: The user attempts to select an invalid difficulty level (e.g., non-existent option, or no selection made).
    Hint: Not selecting an option is also considered an invalid input.
  - Response: The game displays an error message and does not proceed.
- **Invalid Bet Input**
  - Stimulus: The user is at the betting page with a valid game session.
  - Response: The game displays the betting options.
  - Stimulus: The user places an invalid bet amount (e.g., negative number, zero, exceeding available points).
  - Response: The game displays an error message and does not proceed.
- **Invalid Guess Input**
  - Stimulus: The user is at the guessing page with a valid game session.
  - Response: The game shows the make a guess page.
  - Stimulus: The user enters an invalid guess (e.g., number outside 1–100, non-numeric input).
  - Response: The game displays an error message and does not proceed.

Hints

In Robot Framework, always use 4 spaces for indentation (not tabs) to ensure your tests run correctly.

Implementing your own binary search is optional. You can find our implementation in `helpers.robot`.

Binary Search Explanation:

Binary search is an efficient way to guess the number by repeatedly dividing the search range in half.

- Start with the minimum guess as 1 and the maximum guess as 100.
- Calculate the midpoint of the current range.
- If the feedback indicates the number is higher, adjust the minimum guess to the midpoint + 1.
- If the feedback indicates the number is lower, adjust the maximum guess to the midpoint - 1.
- Repeat the process until the correct number is guessed or the maximum number of guesses is reached.

Set Session Parameters Helper Keyword

Utilize the `Set Session Parameters` helper keyword defined in `helpers.robot` to navigate directly to specific parts of the game, bypassing earlier steps. This is useful for efficiently testing invalid inputs without starting from the beginning each time.

Example Usage:

```
Set Session Parameters    TestUser1    easy    5    50    50    10
```

Before Submitting

- Execute the tests and verify that they pass without errors.
- Debug and fix any issues that you had during the test execution.
- Ensure your tests use the provided keywords, follow the scenarios described in the assignment, and are properly structured. Also, use the `Set Session Parameters` keyword to navigate directly to specific parts of the game when required.

Submission Instructions: For this assignment, you will submit two test files, one for each part of the assignment.

- **Part 1:** Implement your test cases into the `studentnumber-part1.robot` file replacing "studentnumber" with your actual student number. This file should include test cases for Normal Game Playthrough (scenario 1), and Repetitive Incorrect Guesses Leading to Loss (scenario 2).
- **Part 2:** Implement your test cases into the `studentnumber-part2.robot` file replacing "studentnumber" with your actual student number. This file should include the test cases for Invalid Name Input (scenario 1), Invalid Difficulty Selection (scenario 2), Invalid Bet Input (scenario 3), and Invalid Guess Input (scenario 4).

For each step, ensure the game provides appropriate feedback for invalid inputs and does not proceed to the next step.

Grading:

- **5-4 points:** The assignment is clear and well-organized. Test cases are correctly implemented, covering both normal and edge cases. Tests include appropriate validation of game responses. `Set Session Parameters` keyword is effectively used to simulate mid-game states, and the tests run without errors. The readability of the tests is high, with clear and concise steps.
- **3-2 points:** The assignment is mostly clear, but some key scenarios or edge cases are missing. Test cases are implemented with minor gaps, and response validation may be incomplete. `Set Session Parameters` is used, but not with sensible arguments. The readability of the tests is adequate, but some parts may be unclear or difficult to follow.
- **1-0 points:** The assignment is incomplete or lacks clarity. Test cases are missing, incorrect, or fail to validate responses properly. The `Set Session Parameters` keyword is either not used or applied incorrectly, and the tests may not run correctly. The readability of the tests is unclear with disorganized steps.

Use of AI and Collaboration with Other Students: Using AI tools to produce an answer for this assignment is not permitted. All parts of the submission must be produced by yourself (apart from files provided in the assignment). Collaboration with other students is not permitted. This is an individual assignment that you must author independently.

AcceptanceTesting2-1.zip 4 October 2024, 1:46 PM

Submission status

Attempt number	This is attempt 1.
Submission status	No submissions have been made yet
Grading status	Not marked
Time remaining	Assignment is overdue by: 155 days 4 hours
Last modified	-

Previous activity
◄ Acceptance Testing 1: rf-katas

Next activity
Acceptance Testing 2.2: Number Guessing Game ►